

Question 1. NumPy.

```
In [3]: # import libraries and create helper functions for displaying matrixes in LaTeX
from IPython.display import display, Math
import numpy as np

def display_matrix(matrix, name):
    matrix_str = "\\begin{bmatrix}"
    if matrix.ndim == 1:
        matrix_str += " & ".join(map(str, matrix)) + " \\\\ "
    else:
        for row in matrix:
            matrix_str += " & ".join(map(str, row)) + " \\\\ "
        matrix_str += "\\end{bmatrix}"
    display(Math(r'\mathbf{' + name + '} = ' + matrix_str))
```

a. Create a NumPy array `np_array1` of integers from 0 to 25 and reshape it into a (5,5) matrix.

```
In [70]: np_array1 = np.arange(25).reshape(5, 5)
display_matrix(np_array1, "A_1")
```

$$\mathbf{A}_1 = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 & 9 \\ 10 & 11 & 12 & 13 & 14 \\ 15 & 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 & 24 \end{bmatrix}$$

b. Create a NumPy array `np_array2` of size 25 of equally spaced numbers from 5.0 to 10.0 (both end points inclusive) and print the array.

```
In [4]: np_array2 = np.linspace(5, 10, 25).reshape(5, 5)
display_matrix(np_array2, "A_2")
```

$$\mathbf{A}_2 = \begin{bmatrix} 5.0 & 5.20833333333333 & 5.41666666666667 & 5.625 & 5.833 \\ 6.04166666666667 & 6.25 & 6.45833333333334 & 6.66666666666667 & \\ 7.08333333333334 & 7.29166666666667 & 7.5 & 7.70833333333334 & 7.916 \\ 8.125 & 8.33333333333334 & 8.54166666666668 & 8.75 & 8.958 \\ 9.16666666666668 & 9.375 & 9.58333333333334 & 9.79166666666668 & \end{bmatrix}$$

This clearly shows too many digits after the decimal point. Write a function called `matrix_round()` that takes in a NumPy array and a desired precision, and returns another array of the same shape with all entries rounded to the desired precision. Remember, the only two inputs to the function should be the NumPy array and the desired precision.

```
In [5]: def matrix_round(array, precision):
        return np.round(array, precision)
np_array2_rounded = matrix_round(np_array2, 3)
display_matrix(np_array2_rounded, "A_2")
```

$$\mathbf{A}_2 = \begin{bmatrix} 5.0 & 5.208 & 5.417 & 5.625 & 5.833 \\ 6.042 & 6.25 & 6.458 & 6.667 & 6.875 \\ 7.083 & 7.292 & 7.5 & 7.708 & 7.917 \\ 8.125 & 8.333 & 8.542 & 8.75 & 8.958 \\ 9.167 & 9.375 & 9.583 & 9.792 & 10.0 \end{bmatrix}$$

c. Read the properties of matrix inversion in the class notes.
Compute the inverse of the matrix V with the following entries
(the linalg package might help here)

```
In [6]: V = np.array([[1,1,1,1,1],
                    [1,2,4,8,16],
                    [1,3,9,27,81],
                    [1,4,16,64,256],
                    [1,5,25,125,625]])
display_matrix(V, "V")
Vinv = np.linalg.inv(V)
display_matrix(Vinv, "V^{-1}")
```

$$\mathbf{V} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 & 16 \\ 1 & 3 & 9 & 27 & 81 \\ 1 & 4 & 16 & 64 & 256 \\ 1 & 5 & 25 & 125 & 625 \end{bmatrix}$$

$$\mathbf{V}^{-1} = \begin{bmatrix} 5.000000000000002 & -9.999999999999996 & 10.0 & -5.0 & 1.0 \\ -6.416666666666668 & 17.833333333333333 & -19.5 & 10.166666666666666 & -2.083333333333333 \\ 2.958333333333333 & -9.833333333333332 & 12.25 & -6.833333333333333 & 1.458333333333333 \\ -0.583333333333333 & 2.166666666666665 & -3.0 & 1.833333333333333 & -0.416666666666667 \\ 0.04166666666666664 & -0.1666666666666666 & 0.25 & -0.1666666666666666 & 0.04166666666666667 \end{bmatrix}$$

Print V^{-1} to 3 decimal places.

```
In [7]: display_matrix(matrix_round(Vinv, 3), "V^{-1}")
```

$$\mathbf{V}^{-1} = \begin{bmatrix} 5.0 & -10.0 & 10.0 & -5.0 & 1.0 \\ -6.417 & 17.833 & -19.5 & 10.167 & -2.083 \\ 2.958 & -9.833 & 12.25 & -6.833 & 1.458 \\ -0.583 & 2.167 & -3.0 & 1.833 & -0.417 \\ 0.042 & -0.167 & 0.25 & -0.167 & 0.042 \end{bmatrix}$$

Compute $V^{-1}V$ and VV^{-1} and print them to 3 decimal places.

```
In [8]: display_matrix(matrix_round(Vinv @ V, 3), "V^{-1}V")
display_matrix(matrix_round(V @ Vinv, 3), "VV^{-1}")
```

$$\mathbf{V}^{-1}\mathbf{V} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ -0.0 & 1.0 & -0.0 & -0.0 & -0.0 \\ 0.0 & 0.0 & 1.0 & -0.0 & -0.0 \\ 0.0 & -0.0 & 0.0 & 1.0 & -0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$$\mathbf{V}\mathbf{V}^{-1} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 & -0.0 \\ -0.0 & 0.0 & 1.0 & 0.0 & -0.0 \\ -0.0 & -0.0 & 0.0 & 1.0 & -0.0 \\ 0.0 & -0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

What is the name given to the matrices $\mathbf{V}^{-1}\mathbf{V}$ and $\mathbf{V}\mathbf{V}^{-1}$?

The matrixes are the Identity Matrix, denoted by I .

What NumPy command is used to generate such matrices?

There are 2 NumPy commands to generate identity matrices: `np.eye()` and `np.identity()`.

Question 2. Solving systems of equations in NumPy.

a. You are given the following system of equations:

$$3x - y + 2z = 3 \quad (1)$$

$$-y + z = 5 \quad (2)$$

$$2x - 3z = -4 \quad (3)$$

Express the system as $\mathbf{A}\mathbf{b} = \mathbf{c}$, where $\mathbf{A}$ is a (3,3) coefficient matrix, $\mathbf{b} = [x, y, z]^T$ is a (3,1) vector of all the unknowns and $\mathbf{c}$ is another (3,1) vector containing the right-hand side of the system of equations.

```
In [9]: A = np.array([[3, -1, 2],
                    [0, -1, 1],
                    [2, 0, -3]])
B = np.array([[ 'x' ], [ 'y' ], [ 'z' ]])
C = np.array([[3], [5], [-4]])
display_matrix(A, "A")
display_matrix(B, "B")
display_matrix(C, "C")
```

$$\mathbf{A} = \begin{bmatrix} 3 & -1 & 2 \\ 0 & -1 & 1 \\ 2 & 0 & -3 \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} 3 \\ 5 \\ -4 \end{bmatrix}$$

b. Print $\mathbf{A}$ and $\mathbf{A}^{-1}$ to 3 decimal places.

```
In [10]: display_matrix(matrix_round(A, 3), "A")
Ainv = np.linalg.inv(A)
display_matrix(matrix_round(Ainv, 3), "A^{-1}")
```

$$\mathbf{A} = \begin{bmatrix} 3 & -1 & 2 \\ 0 & -1 & 1 \\ 2 & 0 & -3 \end{bmatrix}$$

$$\mathbf{A}^{-1} = \begin{bmatrix} 0.273 & -0.273 & 0.091 \\ 0.182 & -1.182 & -0.273 \\ 0.182 & -0.182 & -0.273 \end{bmatrix}$$

How would you use $\mathbf{A}^{-1}$ to solve for the vector $\mathbf{b}$?

To solve for the vector $\mathbf{b}$ use $\mathbf{b} = \mathbf{A}^{-1}\mathbf{c}$. This is because:

1. If $\mathbf{A}\mathbf{b} = \mathbf{c}$
2. Multiply both sides by $\mathbf{A}^{-1}$: $\mathbf{A}^{-1}\mathbf{A}\mathbf{b} = \mathbf{A}^{-1}\mathbf{c}$
3. Since $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ (identity matrix): $\mathbf{I}\mathbf{b} = \mathbf{A}^{-1}\mathbf{c}$
4. Therefore $\mathbf{b} = \mathbf{A}^{-1}\mathbf{c}$

c. What is the value of $\mathbf{b}$?

Print to 3 decimal places.

```
In [11]: B = Ainv @ C
display_matrix(matrix_round(B, 3), "B")
```

$$\mathbf{B} = \begin{bmatrix} -0.909 \\ -4.273 \\ 0.727 \end{bmatrix}$$

d. Overdetermined systems.

When the number of equations is larger than the number of unknowns, the system of equations is said to be "overdetermined." Consider the following system of equations.

$$1x + 4y + 7z = 3 \quad (4)$$

$$2x - 5y + 8z = 5 \quad (5)$$

$$3x + 6y - 9z = -4 \quad (6)$$

$$-x + 3y = 1 \quad (7)$$

$$y = -2 \quad (8)$$

Once again, write this system of equations in the form $A\mathbf{b} = \mathbf{c}$.

```
In [12]: A = np.array([[1, 4, 7],
                      [2, -5, 8],
                      [3, 6, -9],
                      [-1, 3, 0],
                      [0, 1, 0]])
B = np.array(['x'], ['y'], ['z'])
C = np.array([3], [5], [-4], [1], [-2])
display_matrix(A, "A")
display_matrix(B, "B")
display_matrix(C, "C")
```

$$\mathbf{A} = \begin{bmatrix} 1 & 4 & 7 \\ 2 & -5 & 8 \\ 3 & 6 & -9 \\ -1 & 3 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} 3 \\ 5 \\ -4 \\ 1 \\ -2 \end{bmatrix}$$

Is the matrix A invertible?

The matrix A is not invertible because it is not a square matrix.

What happens when you try to invert A ?

An error occurs when you try to invert a non-square matrix.

```
In [13]: try:
          Ainv = np.linalg.inv(A)
        except Exception as e:
          print(f"An error occurred: {e}")
```

An error occurred: Last 2 dimensions of the array must be square

e. In such situations, a special inverse called the "Pseudoinverse" is used. The pseudoinverse of A is denoted by

A^+ . Using the `pinv()` function in the `linalg` package, print $A^+ A$ up to 3 decimal places.

```
In [14]: Apinv = np.linalg.pinv(A)
display_matrix(matrix_round(Apinv, 3), "A^+")
display_matrix(matrix_round(Apinv @ A, 3), "A^+A")
```

$$A^+ = \begin{bmatrix} 0.028 & 0.174 & 0.176 & -0.097 & -0.008 \\ 0.096 & -0.056 & 0.025 & 0.058 & 0.016 \\ 0.069 & 0.026 & -0.034 & 0.018 & 0.005 \end{bmatrix}$$

$$A^+ A = \begin{bmatrix} 1.0 & -0.0 & -0.0 \\ 0.0 & 1.0 & -0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

f. Now, how would you use A^+ to solve $Ab = c$? Is this solution exact or approximate?

The same procedure for the inverse matrix would be applied using the pseudoinverse matrix.*

1. If $Ab = c$
2. Multiply both sides by A^+ : $A^+ Ab = A^+ c$
3. Since $A^+ A \approx I$ (approximates the identity matrix): $Ib \approx A^+ c$
4. Therefore $b \approx A^+ c$

The pseudoinverse gives an exact solution when one exists, such as when the matrix is a square matrix. When no exact solution exists, it provides the best possible approximate solution in terms of least squares. Since there is no exact solution in this example, the solution is approximate.

Question 3. Matplotlib.

Part One: Creating data from an equation

It is important to be able to directly translate a real equation into a plot. Your first task actually is pure numpy, then we will explore how to plot it out with Matplotlib. The [world famous equation](#) from Einstein:

$$E = mc^2$$

Use your knowledge of Numpy to create two arrays: E and m , where m is simply 11 evenly spaced values representing 0 grams to 10 grams. E should be the equivalent energy for the mass. You will need to figure out what to provide for c for the units m/s, a

quick google search will easily give you the answer (let's use the close approximation in our solutions).

```
In [15]: m = np.linspace(0, 10, 11)
display_matrix(matrix_round(m, 3), "m")
```

```
m = [0.0  1.0  2.0  3.0  4.0  5.0  6.0  7.0  8.0  9.0 10.0]
```

```
In [ ]:
```

The array m should look like:

```
[ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
```

```
In [16]: c = 3e8 # speed of light in m/s, using the approximation
E = m * c * c
display_matrix(np.array([f"{x:.1e}" for x in E]), "E")
```

```
E = [0.0e+00  9.0e+16  1.8e+17  2.7e+17  3.6e+17  4.5e+17  5.4e+17  6.3e+17  7.2e+17  8.1e+17  9.0e+17]
```

```
In [ ]:
```

The array E should look like:

```
[0.0e+00 9.0e+16 1.8e+17 2.7e+17 3.6e+17 4.5e+17 5.4e+17 6.3e+17 7.2e+17 8.1e+17 9.0e+17]
```

Part Two: Plotting $E = mc^2$

Now that we have the arrays E and m, we can plot this to see the relationship between Energy and Mass.

Import what you need from Matplotlib to plot out graphs.

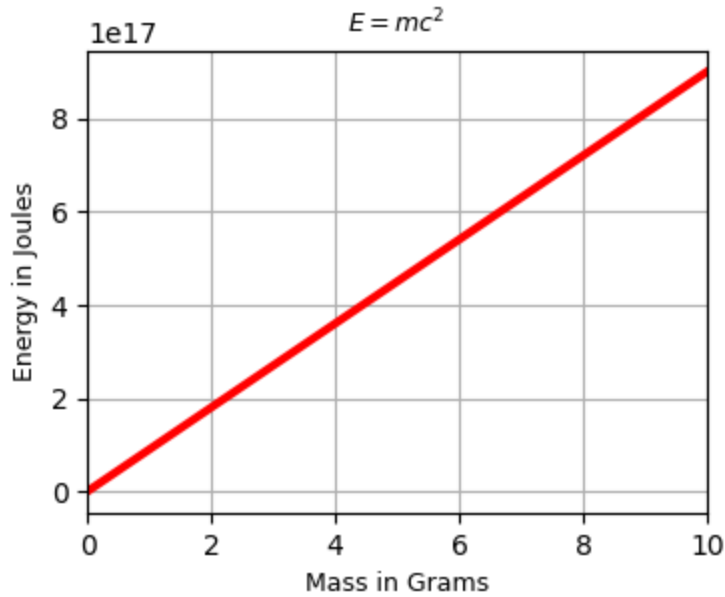
TASK: Recreate the plot shown below which maps out $E = mc^2$ using the arrays we created in the previous task. Note the labels, titles, color, and axis limits.

You should attempt to re-create each major component. Better recreations will receive higher points.

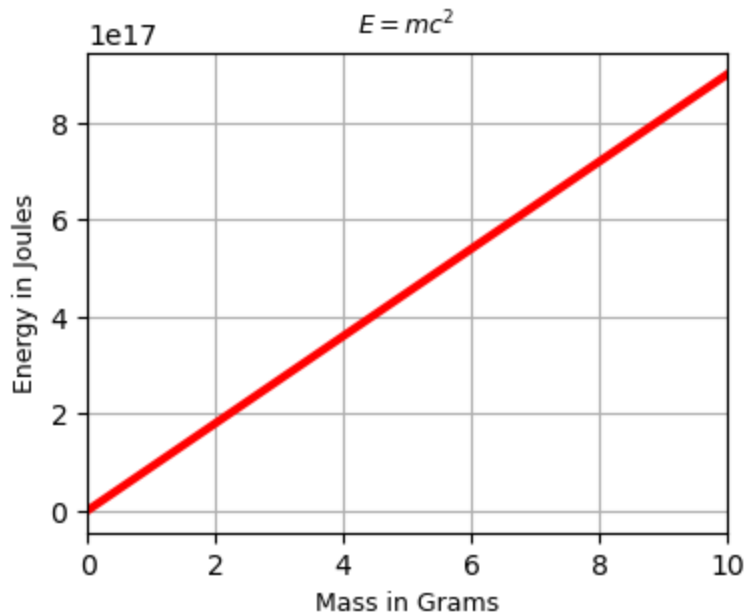
Use the figure and axis objects to complete the task.

```
In [17]: import matplotlib.pyplot as plt

plt.figure(figsize=(4, 3))
plt.plot(m, E, color='red', linewidth=3.0)
plt.xlabel('Mass in Grams', fontsize=9)
plt.ylabel('Energy in Joules', fontsize=9)
plt.title(r'$E = mc^2$', fontsize=9)
plt.xlim(0,10)
plt.grid(visible=True)
plt.show()
```



In []:



In []:

Matplotlib Exercise Project (part 2)

As always, you should attempt to re-create each major component. **Better recreations will receive higher points.**

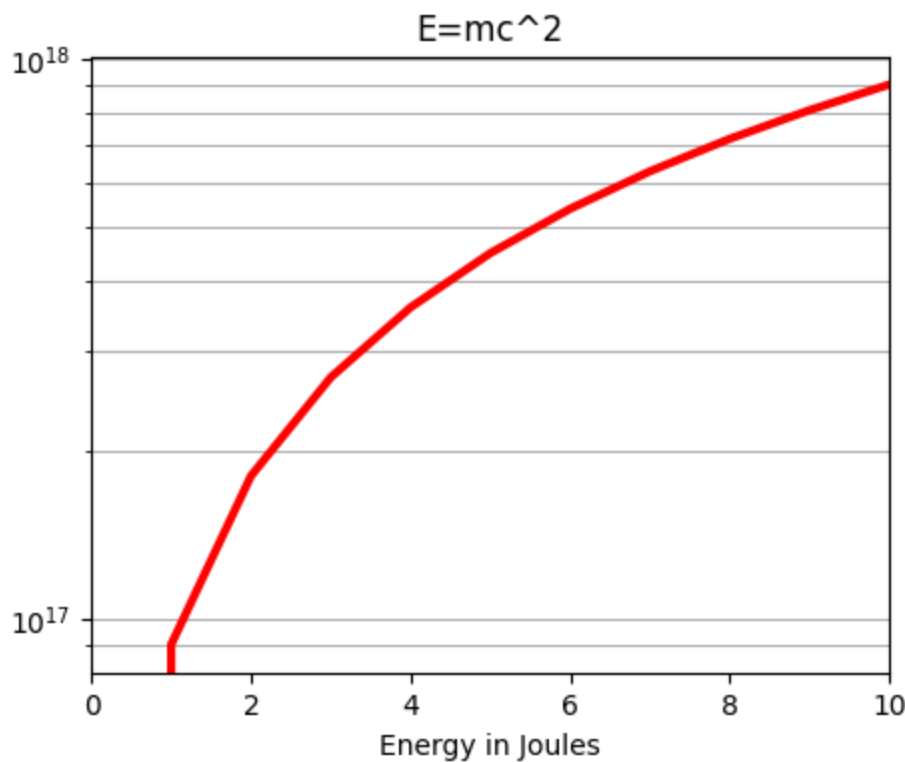
Do not run the cells right above the plots or they will disappear.

1. We will continue with our plotting of the $E = mc^2$ equation from the previous homework. Can you figure out how to plot the *mass vs Energy* plot on a *logarithmic*

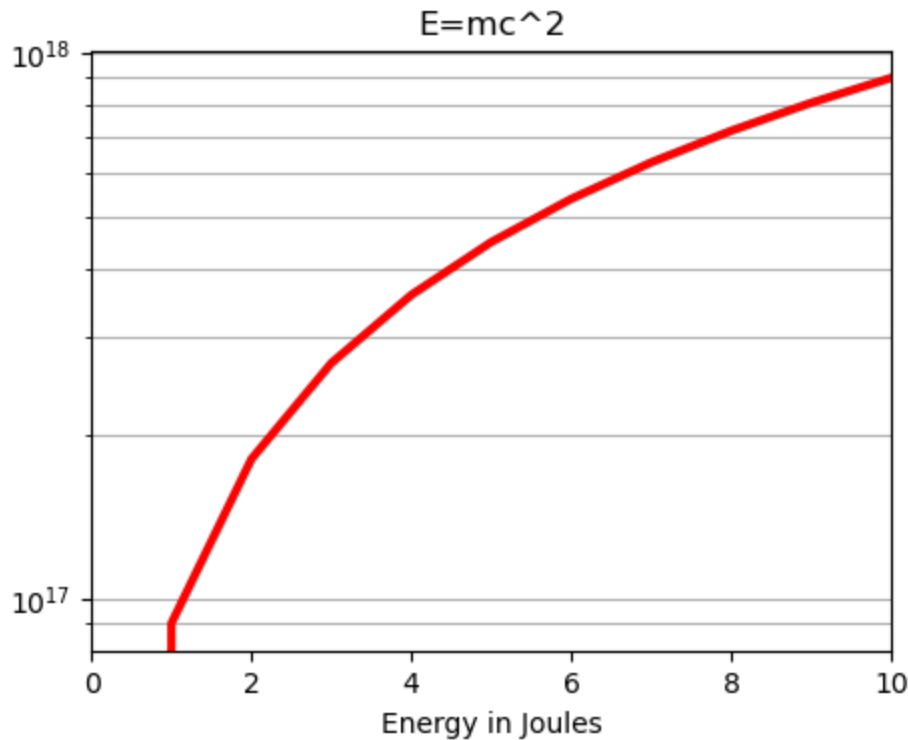
scale on the y axis? Place a grid along the y axis ticks as well. The plot shown here only required minor modifications of the code for the changes.

- You will have to carefully review the options associated with the `plt.grid()` command for this question.

```
In [18]: plt.figure(figsize=(5.15, 4))
plt.plot(m, E, color='red', linewidth=3.0)
plt.xlabel('Energy in Joules', fontsize=10) # This label is wrong!! I'm only
plt.title("E=mc^2", fontsize=12)
plt.xlim(0,10)
plt.yscale('log')
plt.grid(False)
plt.gca().yaxis.grid(True, which='both')
plt.show()
```



In []:



2. Assume that the data below shows the interest paid for a US Treasury bond for a certain contract length. The labels list shows the corresponding contract length per index position. * Labels = Contract leng

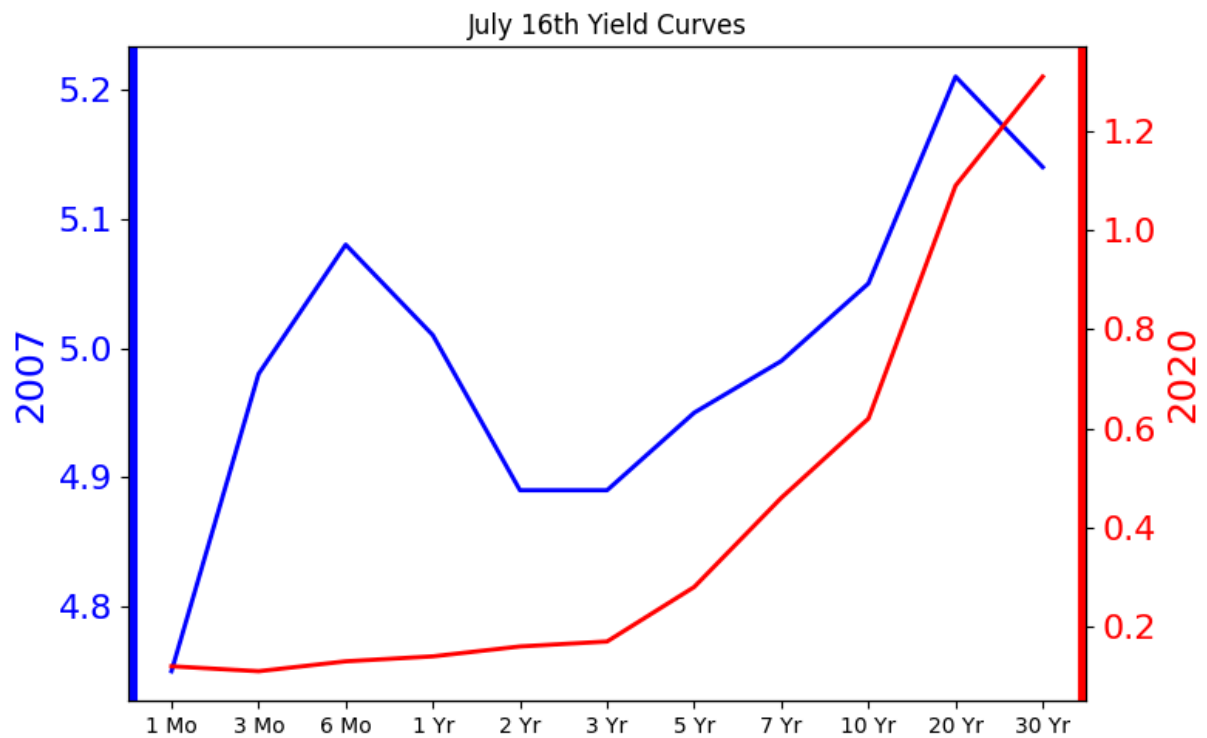
- h Y axis = interest p

Plot the yield curves for both 2007 and 2020 as shown in the figure.aid

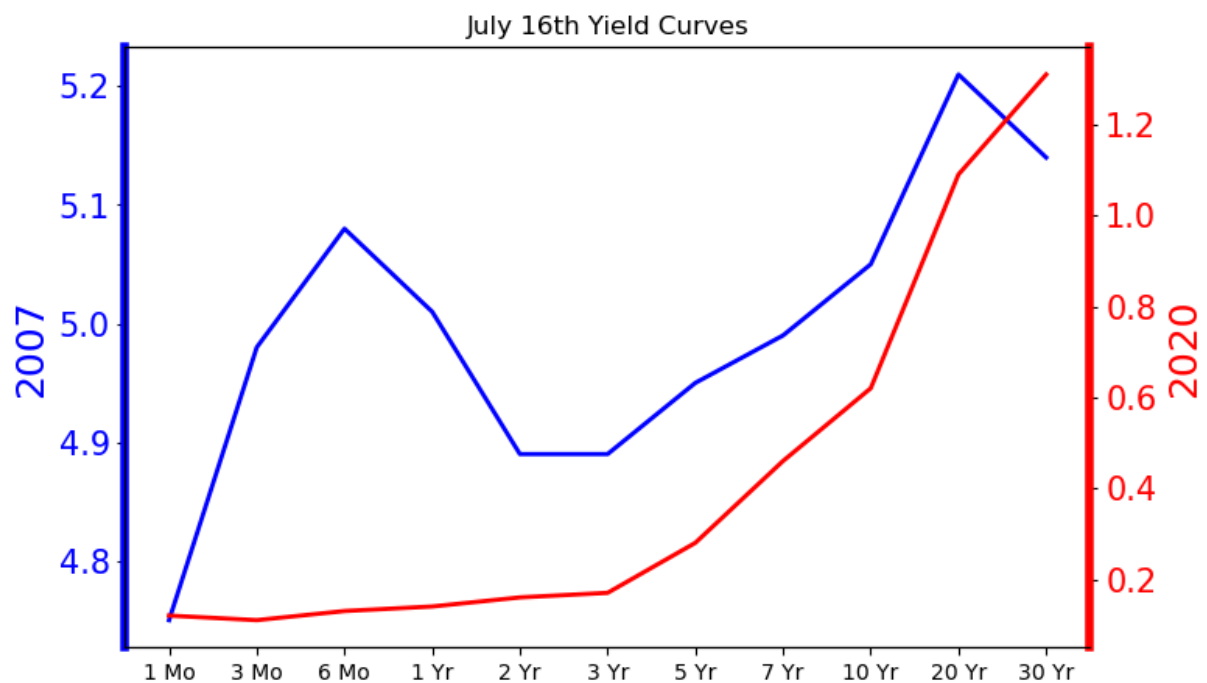
```
In [19]: labels = ['1 Mo', '3 Mo', '6 Mo', '1 Yr', '2 Yr', '3 Yr', '5 Yr', '7 Yr', '10 Yr', '2 Yr']
july16_2007 = [4.75, 4.98, 5.08, 5.01, 4.89, 4.89, 4.95, 4.99, 5.05, 5.21, 5.14]
july16_2020 = [0.12, 0.11, 0.13, 0.14, 0.16, 0.17, 0.28, 0.46, 0.62, 1.09, 1.31]
```

```
In [20]: fig, ax1 = plt.subplots(figsize=(8, 5))
ax2 = ax1.twinx()
ax1.plot(labels, july16_2007, 'b-', label='July 16, 2007', linewidth=2)
ax2.plot(labels, july16_2020, 'r-', label='July 16, 2020', linewidth=2)
ax1.set_ylabel('2007', color='blue', fontsize=18)
ax2.set_ylabel('2020', color='red', fontsize=18)
ax1.tick_params(axis='y', labelcolor='blue', labelsz=16)
ax2.tick_params(axis='y', labelcolor='red', labelsz=16)
ax = plt.gca()
xmin, xmax = ax.get_xlim()
plt.axvline(x=xmin, color='blue', linestyle='--', linewidth=8)
plt.axvline(x=xmax, color='red', linestyle='--', linewidth=8)
```

```
plt.title("July 16th Yield Curves")
fig.tight_layout()
plt.show()
```



In []:



3. This problem will test your ability to generate subplots of varying sizes and types.
 - A. Data: generate a 1D array `x` comprising 100 equally spaced points between 0 and 10.

- B. Using `x` as input, generate y-axis data corresponding to the following three functions: $\sin()$, $e^()$, and $\log(\cdot + 1)$
- C. Use the `subplot2grid()` method to reproduce the plot below as closely as possible.

```
In [21]: x = np.linspace(0, 10, 100)
sin = np.sin(x)
exp = np.exp(x)
log = np.log(x+1)

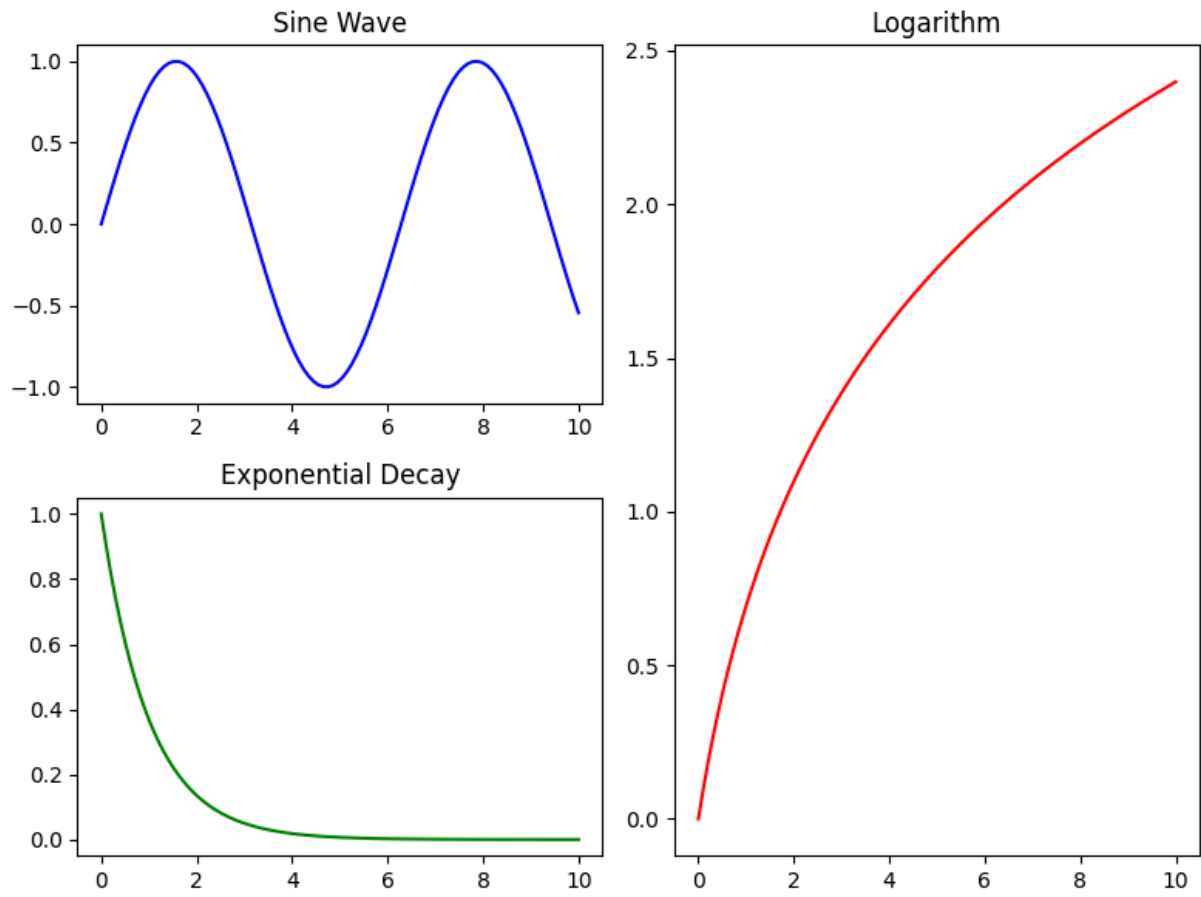
fig = plt.figure(figsize=(8, 6))
ax1 = plt.subplot2grid((2, 2), (0, 0))
ax2 = plt.subplot2grid((2, 2), (1, 0))
ax3 = plt.subplot2grid((2, 2), (0, 1), rowspan=2)

ax1.plot(x, sin, 'b-')
ax1.set_title('Sine Wave')

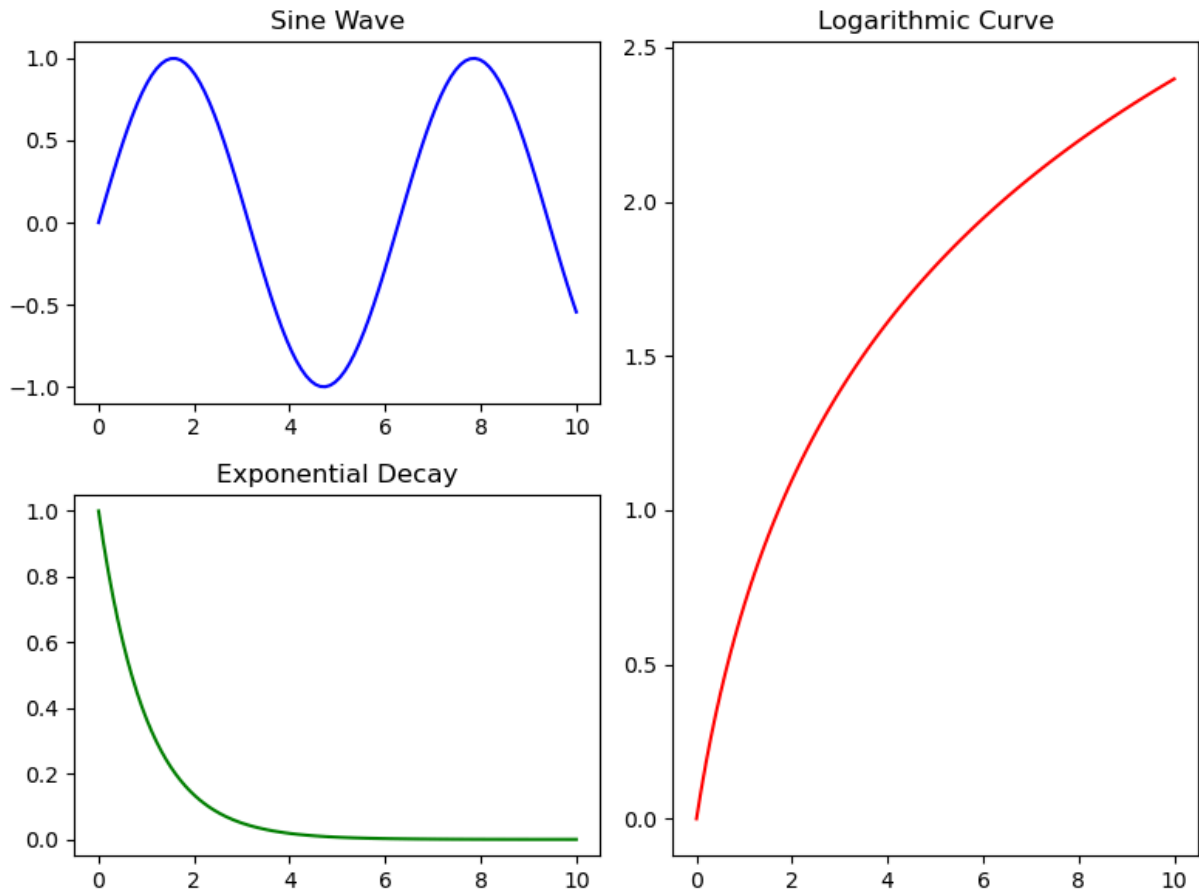
ax2.plot(x, 1/exp, 'g-')
ax2.set_title('Exponential Decay')

ax3.plot(x, log, 'r-')
ax3.set_title('Logarithm')

fig.tight_layout()
plt.show()
```



In []:



In []:

4. Cross Validation.

(References: [GBC16] Sec. 5.2 and 5.3, [BB22] Sec. 1.2.4 – 1.2.6)

a. Using the function $\mathcal{L}(\mathbf{w}, \lambda)$ discussed in class, explain why the optimal regularization coefficient, λ^* cannot be learned from $\mathcal{D}_{train}$, and requires a separate validation set. Assume we're using 2-norm regularization. Recall that we defined

$$\mathcal{L}(\mathbf{w}, \lambda) = \frac{1}{\mathcal{N}_{train}} \sum_{i=1}^{\mathcal{N}_{train}} (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

The optimal regularization coefficient cannot be learned from the training dataset alone because such a process will yield $\lambda^* = 0$, which defeats the purpose of regularization. Regularization is specifically designed to prevent overfitting to the training data by adding the penalty term. If the optimal regularization coefficient is learned from the training data only, then a better fit will always be achieved with less regularization. A better fit to the training data results in lower training error, but this counteracts the goal of regularization to prevent the overfitting. By using a validation set or employing cross-

validation, whenever overfitting occurs during training, it will be counteracted by regularization during validation.

b. Consider the steps involved in cross-validation for the specific case of polynomial regression.

- Assume the dataset contains 2-dimensional features (i.e., $k = 2$), and we're using polynomials of degree $d = 3$. How many terms will the predictor $h(\mathbf{w}, x)$ have?

The predictor $h(\mathbf{w}, x)$ will have $\frac{(k+d)!}{k!d!} = \frac{(2+3)!}{2!3!} = \frac{120}{12} = 10$ terms.

- Write the full form of the predictor $\hat{y} = h(\mathbf{w}, x)$ showing all the constant, linear, quadratic, and cubic terms (including interaction terms).

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 + w_6x_1^3 + w_7x_2^3 + w_8x_1^2x_2 + w_9x_1x_2^2$$

- What happens to the λ^* that is found during the cross-validation process? With what data and loss function are the final weights computed before the testing phase begins?

The λ^* that is found during the cross-validation process is kept fixed when cross-validation is complete. Since it is now the *optimal* regularization parameter, it should not be discarded or changed for the final training. The final weights, $\mathbf{w}^*$ are computed in the final training phase using all of the training data (combining all of the folds) and the λ^* found from the cross-validation process. The complete loss function that is used to compute the $\mathbf{w}^*$ is

$$\mathcal{L}(\mathbf{w}) = \frac{1}{\mathcal{N}_{train}} \sum_{i=1}^{\mathcal{N}_{train}} (y_i - \hat{y}_i)^2 + \lambda^* \|\mathbf{w}\|_2^2$$